

DEPARTMENT OF COMPUTER SCIENCE



University of
Reading

CS1IP: Imperative Programming

Big-O Notation and Search Algorithms

Learning Goals

- What Algorithms Time Complexity Using Big-O Notation?
- How to measure time of programs in Java?
- What is Linear Search?
- What is Binary Search?

Complexity using Big-O Notation

What is a program?

A sequence of **instructions** that is **executed** by a **computer**.

When number of instructions grown with the problem complexity.

One way to know how much a program is complex is by ***understanding its Algorithm Complexity***.

Why complexity matter?

Case Study: Knight Capital Group (2012)

Knight Capital Group, a major player in stock trading, lost \$460 million in a matter of 45 minutes due to a software bug stemming from poor code complexity management.



What is an Algorithm?

An algorithm is a **step-by-step procedure** or set of rules designed to perform a task or solve a problem.

Example

To find the maximum number in an array:

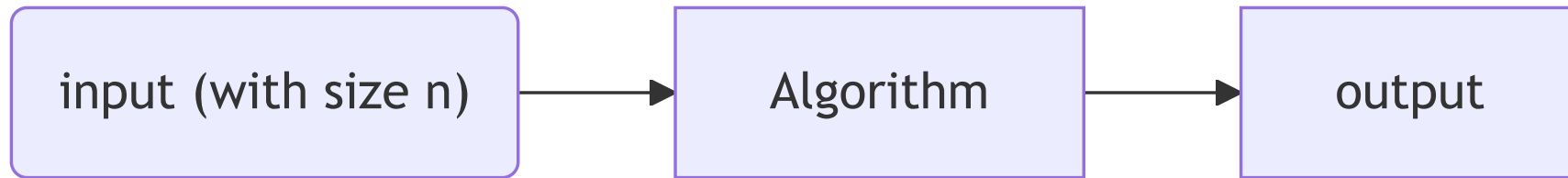
1. Initialize a variable, `max`, to the first element of the array.
2. Iterate through the array and compare each element with `max`.
3. Update `max` if the current element is greater.
4. Return `max` as the result.

Characteristics of a Good Algorithm

- **Finite:** It must terminate after a finite number of steps.
- **Definitive:** Each step should be clear and unambiguous.
- **Input/Output:** It should have specified inputs and produce outputs.
- **Effective:** It should be efficient in terms of time and space.

Introduction to Time Complexity

It is a way to analyze an algorithm's efficiency by measuring the amount of time it takes to run for input.



- **Why is time complexity important?**
 - Helps in understanding how an algorithm scales with input size.
 - Crucial for optimising performance.

Analyzing Algorithms

- **Best Case:** The scenario where the algorithm performs the least number of operations.
- **Worst Case:** The scenario where the algorithm performs the greatest number of operations (Big-O focuses on this).

What is Big-O Notation?

It is a mathematical notation used to describe the worst-case scenario of how an algorithm performs in relation to the input size.

- **Purpose:** To classify algorithms based on how their run time or space requirements grow as the input size grows.
- **Syntax:** $O(\dots)$

$O(1)$: Constant Time

Example: Accessing an element by index

```
int getFirstElement(int[] arr) {  
    return arr[0];  
}
```

$O(n)$: Linear Time

Example: Summing all elements in an array

```
int sumArray(int[] arr) {  
    int sum = 0;  
    for (int i = 0; i < arr.length; i++) {  
        sum += arr[i];  
    }  
    return sum;  
}
```

$O(n^2)$: Quadratic Time

Example: Nested loops

```
void printPairs(int[] arr) {  
    for (int i = 0; i < arr.length; i++) {  
        for (int j = 0; j < arr.length; j++) {  
            System.out.println(arr[i] + ", " + arr[j]);  
        }  
    }  
}
```

$O(n^2)$: Quadratic Time

Example: Not Nested loops

```
int sum = 0;
for (int i = 0; i < arr.length; i++) {
    sum += arr[i];
}
for (int i = 0; i < arr.length; i++) {
    for (int j = 0; j < arr.length; j++) {
        System.out.println(arr[i] + ", " + arr[j]);
    }
}
```

$O(\log n)$: Logarithmic Time

Example: count the number of digits in an integer

- the number of digits in a number n is approximately $\log_{10}(n)$
- the time complexity of this method is $O(\log n)$

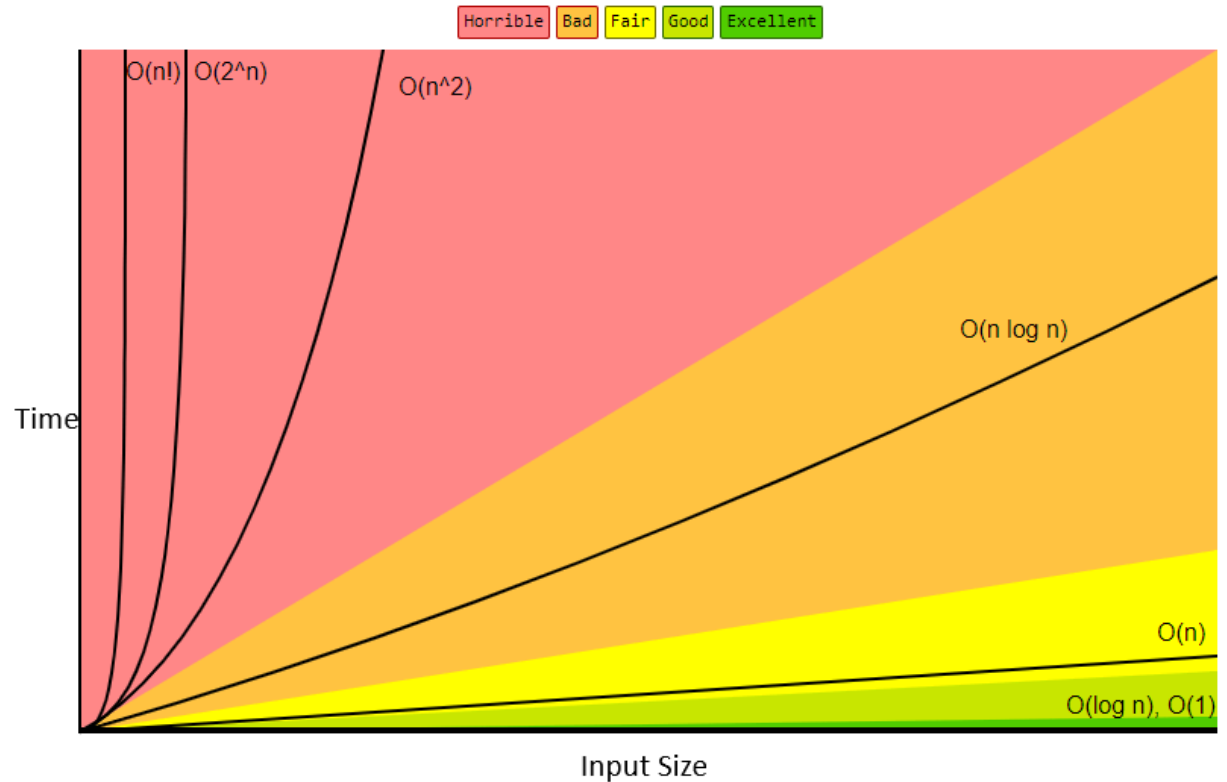
$O(\log n)$: Logarithmic Time

Example: count the number of digits in an integer

```
int countDigits(int n) {  
    int count = 0;  
    while (n > 0) {  
        n = n / 10; // Divide the number by 10  
        count++;  
    }  
    return count;  
}  
System.out.println(countDigits(123456)); // Output: 6  
System.out.println(countDigits(1000));  // Output: 4
```

Common Big-O Notations

- $O(1)$: Constant time
- $O(\log n)$: Logarithmic time
- $O(n)$: Linear time
- $O(n \log n)$: Log-linear time
- $O(n^2)$: Quadratic time
- $O(2^n)$: Exponential time
- $O(n!)$: Factorial time



Optimizing Code for Better Complexity

- Common strategies:
 - Reducing nested loops
 - Efficient data structures
 - Avoiding unnecessary computations
- Example: Refactoring a quadratic algorithm to linear

in-class quiz 1

Week9: Is the overall time complexity of the program $O(n^5)$?

```
for (int i = 0; i < n * n; i++) {  
    System.out.println("Outer loop iteration: " + i);  
}  
  
for (int j = 0; j < n * n * n; j++) {  
    System.out.println("Inner loop iteration: " + j);  
}
```

- Yes, because the loops multiply their complexities.
- No, because the loops are not nested.
- Yes, because the sum of $O(n^2)$ and $O(n^3)$ is equivalent to $O(n^5)$.
- No, because the overall complexity is $O(n^3)$.

Examples of Algorithms analysis

- Comparing two algorithms for Check Prime Number
- Comparing two algorithms for Power Calculation
- Comparing two algorithms for Fibonacci

Comparing Two Algorithms for Checking Prime Numbers

Algorithm 1: Brute Force

```
boolean isPrime(int n) {  
    if (n <= 1) return false;  
    for (int i = 2; i < n; i++) {  
        if (n % i == 0) return false;  
    }  
    return true;  
}
```

- Time Complexity: $O(n)$.

Comparing Two Algorithms for Checking Prime Numbers

Algorithm 2: Optimized Approach

```
boolean isPrime(int n) {  
    if (n <= 1) return false;  
    for (int i = 2; i <= Math.sqrt(n); i++){  
        if (n % i == 0) return false;  
    }  
    return true;  
}
```

 Which is its complexity?

Comparing Two Algorithms for Checking Prime Numbers

Algorithm 2: Optimized Approach

- Time Complexity: $O(\sqrt{n})$.
 - Instead of checking all numbers up to $n - 1$, it only checks divisors up to $\sqrt{n}$, as any larger factor would necessarily pair with a smaller one already checked.
- This significantly reduces the number of iterations, making the complexity proportional to the square root of n .

Comparing two algorithms for Power Calculation

Algorithm 1: Iterative Approach

```
int power(int base, int exp) {  
    int result = 1;  
    for (int i = 0; i < exp; i++) {  
        result *= base;  
    }  
    return result;  
}
```

- Time Complexity: $O(n)$, where n is the exponent.

Comparing Two Algorithms for Power Calculation

Algorithm 2: Exponentiation by Squaring

We can reduce the calculation by base^{exp} by reducing the exponent by half

$$\text{base}^{\text{exp}} = \left(\text{base}^{\text{exp}/2}\right) \cdot \left(\text{base}^{\text{exp}/2}\right)$$

Each recursive call divides the exponent in half (for even cases), reducing the number of multiplications exponentially, leading to $O(\log \text{exp})$.

- Case 1: $\text{base} = 2, \text{exp} = 8$

$$2^8 = (2^4) \cdot (2^4)$$

$$2^4 = (2^2) \cdot (2^2)$$

$$2^2 = (2^1) \cdot (2^1)$$

$$2^1 = 2$$

- Case 2: $\text{base} = 3, \text{exp} = 5$

$$3^5 = 3 \cdot (3^4)$$

$$3^4 = (3^2) \cdot (3^2)$$

$$3^2 = (3^1) \cdot (3^1)$$

$$3^1 = 3$$

Comparing Two Algorithms for Power Calculation

Algorithm 2: Exponentiation by Squaring

```
int power(int base, int exp) {  
    if (exp == 0) return 1;  
    if (exp % 2 == 0) {  
        int half = power(base, exp / 2);  
        return half * half;  
    } else {  
        return base * power(base, exp - 1);  
    }  
}
```

 Which is its complexity?

Comparing Two Algorithms for Power Calculation

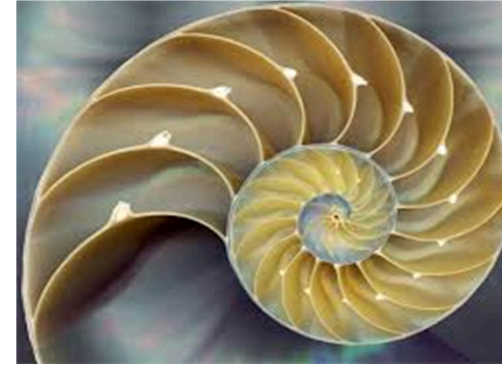
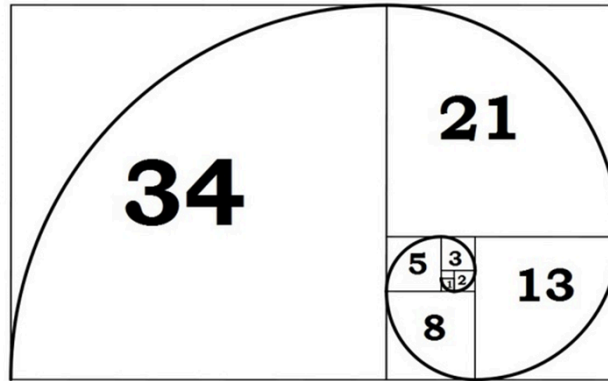
Algorithm 2: Exponentiation by Squaring

- **Time Complexity:** $O(\log n)$, due to halving the exponent.
 - The exponentiation is reduced by half in each recursive step (due to $\text{exp} / 2$, minimizing the number of calls).
 - This results in a logarithmic growth of the recursion depth relative to the exponent, exp .

Comparing two algorithms for Fibonacci Numbers

- Recursive Fibonacci
- Fibonacci with Memorization

What is Fibonacci Numbers?



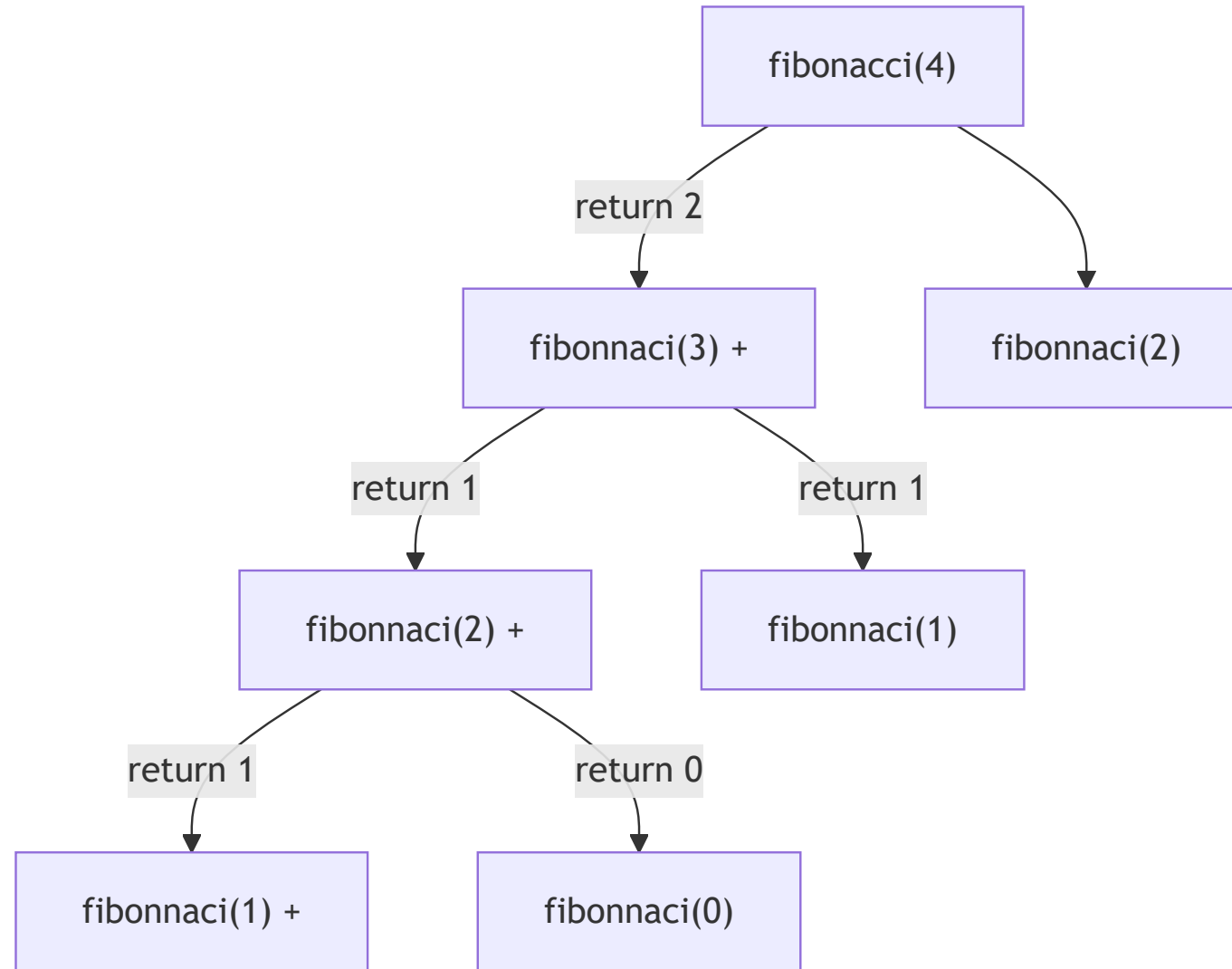
Mathematical Definition

$$fib(n) = 0 \text{ if } n = 0$$

$$fib(n) = 1 \text{ if } n = 1$$

$$fib(n) = fib(n - 1) + fib(n - 2) \text{ if } n > 1$$

What is Fibonacci Numbers?



Recursive Fibonacci

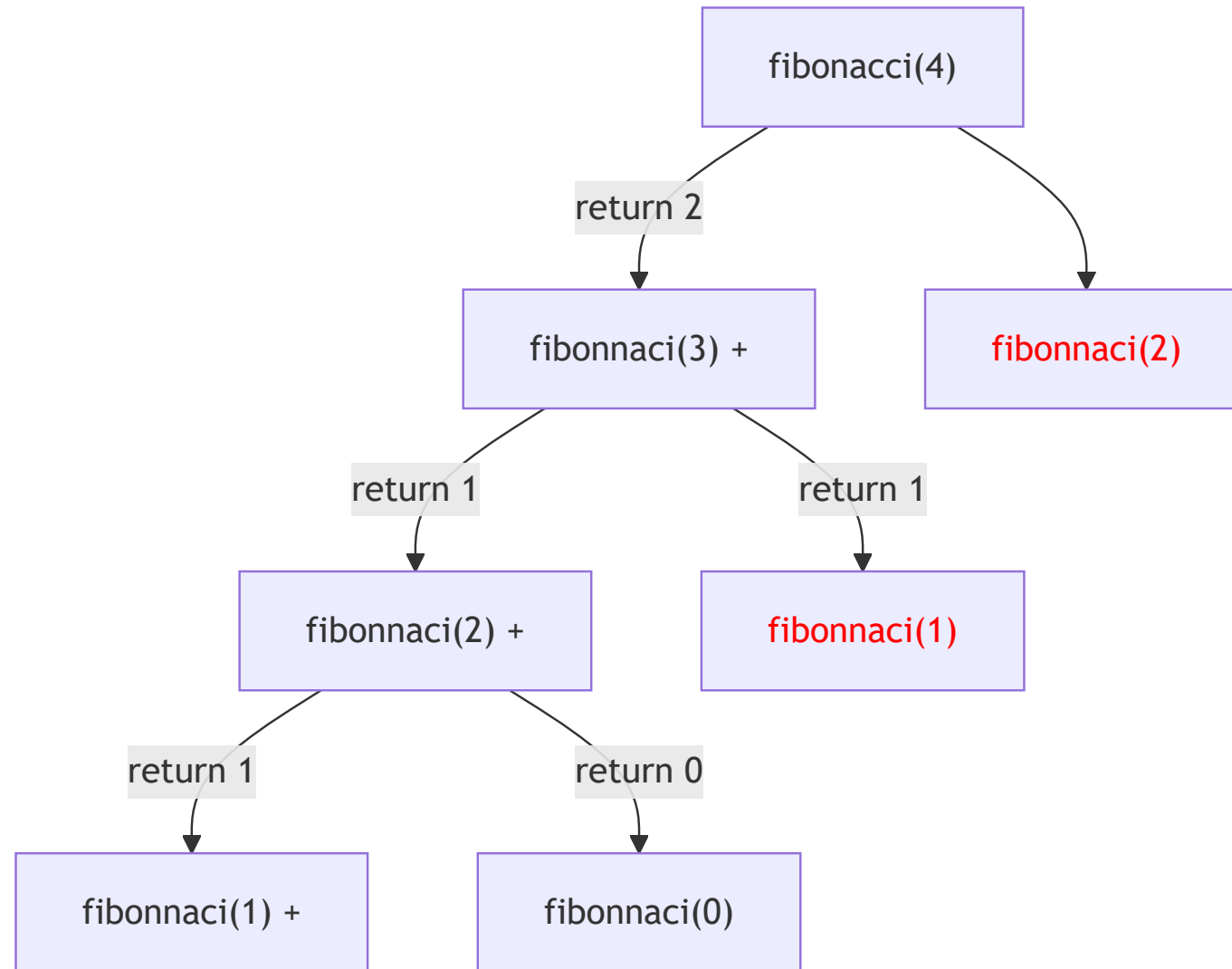
```
int fib(int n) {  
    if (n <= 1) {  
        return n; // Base case  
    }  
    return fibonacci(n - 1) + fibonacci(n - 2);  
}
```

 Which is its complexity?

Recursive Fibonacci

- **Time Complexity:** $O(2^n)$
 - i. Each call to `fib(n)` generates two recursive calls for `fib(n-1)` and `fib(n-2)`, creating a binary tree of calls with exponential growth.
 - ii. The tree has approximately $O(2^n)$ nodes, leading to an exponential number of redundant computations for the same values.
- It is easy to implement but suffers from exponential time complexity due to redundant calculations.

Fibonacci with Memorization



Fibonacci with Memorization

```
int[] memo;

int fib(int n) {
    if (memo[n] != -1) {
        return memo[n]; // Return from memo
    }
    if (n <= 1) {
        return n; // Base case
    }
    memo[n] = fib(n - 1) + fib(n - 2); // Store on memo
    return memo[n];
}
```

 Which is its complexity?

Fibonacci with Memorization

- **Time Complexity:** $O(n)$
 - Each Fibonacci value is computed only once and stored in the memorization array, avoiding redundant calculations.
 - The algorithm processes n unique states, resulting in linear growth proportional to the input size.
- It significantly reduces the number of computations by storing previously calculated results, making it much more efficient.

in-class quiz 2

Week9: What is the primary purpose of Big-O Notation in algorithm analysis?

- To describe the best-case performance of an algorithm.
- To classify algorithms based on how their runtime grow as input size increases.
- To compare the programming languages used to implement an algorithm.
- To identify bugs in an algorithm.

Time Measurement

Why Time Measurement Matters?

- It is crucial for understanding and improving the efficiency of algorithms.
 - especially as the input size grows.
- It impacts user experience, resource management, and scalability in real-world applications.
- By measuring time, developers can identify the **slowest parts** of an algorithm.
 - Performance analysis through time measurement can guide optimizations and improvements.

How do time Measurement in Java?

```
long startTime = System.currentTimeMillis();  
algorithm(input);  
long time = System.currentTimeMillis() - startTime;
```

- Java Doc for `System.currentTimeMillis()`

Search Algorithms

Search Algorithms

Search algorithms are methods for finding a specific element or solving a problem by exploring the given data set.

Where is the 42?

i	0	1	2	3	4
arr[i]	0	42	2	3	4

Classification

Unordered Searches:

- Linear Search

Ordered Searches:

- Binary Search

Linear Search

Introduction to Linear Search

What is Linear Search?

- A simple search algorithm.

Algorithm

1. Check each element in a list sequentially until found or the list ends.

Linear Search: How it Works

Example

- Given an array: [3, 5, 7, 2, 8]
- **Target:** Find 7.
- **Steps:**
 - Check index 0 : 3 (not a match)
 - Check index 1 : 5 (not a match)
 - Check index 2 : 7 (match found!)

```
int linearSearch(ArrayList<Integer> list, int target) {  
    for (int i = 0; i < list.size(); i++) {  
        if (list.get(i) == target) {  
            return i;  
        }  
    }  
    return -1; // Return -1 if target is not found  
}
```

Linear Search Time Complexity

```
int linearSearch(ArrayList<Integer> list, int target) {  
    for (int i = 0; i < list.size(); i++) {  
        if (list.get(i) == target) {  
            return i;  
        }  
    }  
    return -1; // Return -1 if target is not found  
}
```

- **Best Case:** $O(1)$ when first element is the target
- **Worst Case:** $O(n)$ when target is the last element or not present

Binary Search

What is Binary Search?

A search algorithm within a sorted array.

Key Concepts

- **Sorted Array:** it requires the array to be sorted.
- **Divide and Conquer:** The array is divided into two halves to reduce the search space.
- **Efficiency:** The time complexity is $O(\log n)$, where n is the number of elements.

Binary Search Algorithm

- **Step 1: Select the middle element**
- **Step 2: Compare:**
 - If the middle element is the target, return the index.
 - If the target is less, narrow the search to the left half.
 - If the target is greater, narrow the search to the right half.
- **Step 3: Repeat:**
 - Continue until the element is found or the array is exhausted.

Binary Search Example

- **Sorted Array:** [2, 5, 8, 12, 16, 23, 38, 56, 72, 91]
- **Target:** 23
- **Step1: Select middle element:** middle element is 16

```
[2, 5, 8, 12, 16, 23, 38, 56, 72, 91]
               ^
```

- **Step 2: Compare:** $23 > 16$, yes, so it could be in the right part
- **Step 3: Repeat:** Search right half

Binary Search Example

- Spited array: [23, 38, 56, 72, 91]
- Target: 23
- Step1: Select middle element: middle element is 56

```
[23, 38, 56, 72, 91]  
          ^
```

- Step 2: Compare: $23 > 56$, no, so it could be in the left part
- Step 3: Repeat: Search left half

Binary Search Example

- Spited array: [23, 38]
- Target: 23
- Step1: Select middle element: middle element is 23

```
[23, 38]  
  ^
```

- Step 2: Compare: $23 == 23$, yes, you found the index

binarySearch()

```
...
int binarySearch(ArrayList<Integer> list, int target, int left, int right) {
    if (left > right) {
        return -1; // Base case: target not found
    }

    int mid = left + (right - left) / 2;

    if (list.get(mid) == target) {
        return mid; // Target found at index mid
    } else if (target > list.get(mid)) {
        return binarySearch(list, target, mid + 1, right); // Search in the right half
    } else {
        return binarySearch(list, target, left, mid - 1); // Search in the left half
    }
}
```

- `left` : Starting index of the search space
- `right` : Ending index of the search space
- `mid` : Middle index to compare the target against

binarySearch()

```
ArrayList<Integer> numbers = new ArrayList<>(List.of(2, 5, 8, 12, 16, 42, 38, 56, 72, 91));
int target = 42;
Collections.sort(numbers); // binarySearch required a sorted array.
int result = binarySearch(numbers, target, 0, numbers.size() - 1);

if (result == -1) {
    System.out.println("Element not present in the ArrayList.");
} else {
    System.out.println("Element found at index: " + result);
}
```

Binary Search Time Complexity

- **Best Case:** $O(1)$ when target is in the middle
- **Worst Case:** $O(n)$ when target is the last from the subdivisions

Comparing Linear and Binary Search Algorithms

Comparing Linear and Binary Search Algorithms

Feature	Linear Search	Binary Search
Input Requirement	No specific order	Requires sorted input
Best Case	$O(1)$	$O(1)$
Worst Case	$O(n)$	$O(\log n)$

- **Linear Search** is simpler but lowest for large data sets.
- **Binary Search** is more efficient for sorted arrays due to its divide-and-conquer approach. Required the array be sorted

Comparing Linear and Binary Search Algorithms

We will compare the performance of **Linear Search** and **Binary Search** for arrays of sizes `10` , `100` , and `10000` .

- `generateInputFile(int size, String filename)`
- `measureLinearSearch(String file)`
- `measureBinarySearch(String file)`

generateSearchFile(int size, String filename)

```
void generateSearchFile(int size, String filename) throws IOException {
    Random random = new Random();
    FileWriter writer = new FileWriter(filename);
    for (int i = 1; i < size*1000; i++) {
        int num;
        do {
            num = random.nextInt(1000);
        } while (num == 42);
        writer.write( + "\n");
    }
    writer.write("42\n");
    writer.close();
}

// Generate search input
generateSearchFile(10, "input10.txt");
generateSearchFile(100, "input100.txt");
generateSearchFile(10000, "input10000.txt");
```

measureLinearSearch(String file)

```
long measureLinearSearch(String filename, int target) throws IOException {  
    var array = new ArrayList<Integer>();  
    Scanner sc = new Scanner(new File(filename));  
    while (sc.hasNextInt()) {  
        array.add(sc.nextInt());  
    }  
    long startTime = System.currentTimeMillis();  
    linearSearch(array, target);  
    return System.currentTimeMillis() - startTime;  
}
```

measureBinarySearch(String file)

```
long measureBinarySearch(String filename, int target) throws IOException {  
    var array = new ArrayList<Integer>();  
    Scanner sc = new Scanner(new File(filename));  
    while (sc.hasNextInt()) {  
        array.add(sc.nextInt());  
    }  
    Collections.sort(array); // binarySearch required a sorted array.  
    long startTime = System.currentTimeMillis();  
    binarySearch(array, target, 0, array.size() - 1);  
    return System.currentTimeMillis() - startTime;  
}
```

Measure Linear Search and Binary Search

```
System.out.println("Linear Search:")
measureLinearSearch("input10.txt", 42);
measureLinearSearch("input100.txt", 42);
measureLinearSearch("input10000.txt", 42);
System.out.println("Binary Search:")
measureBinarySearch("input10.txt", 42);
measureBinarySearch("input100.txt", 42);
measureBinarySearch("input10000.txt", 42);
```

Linear Search:

1

4

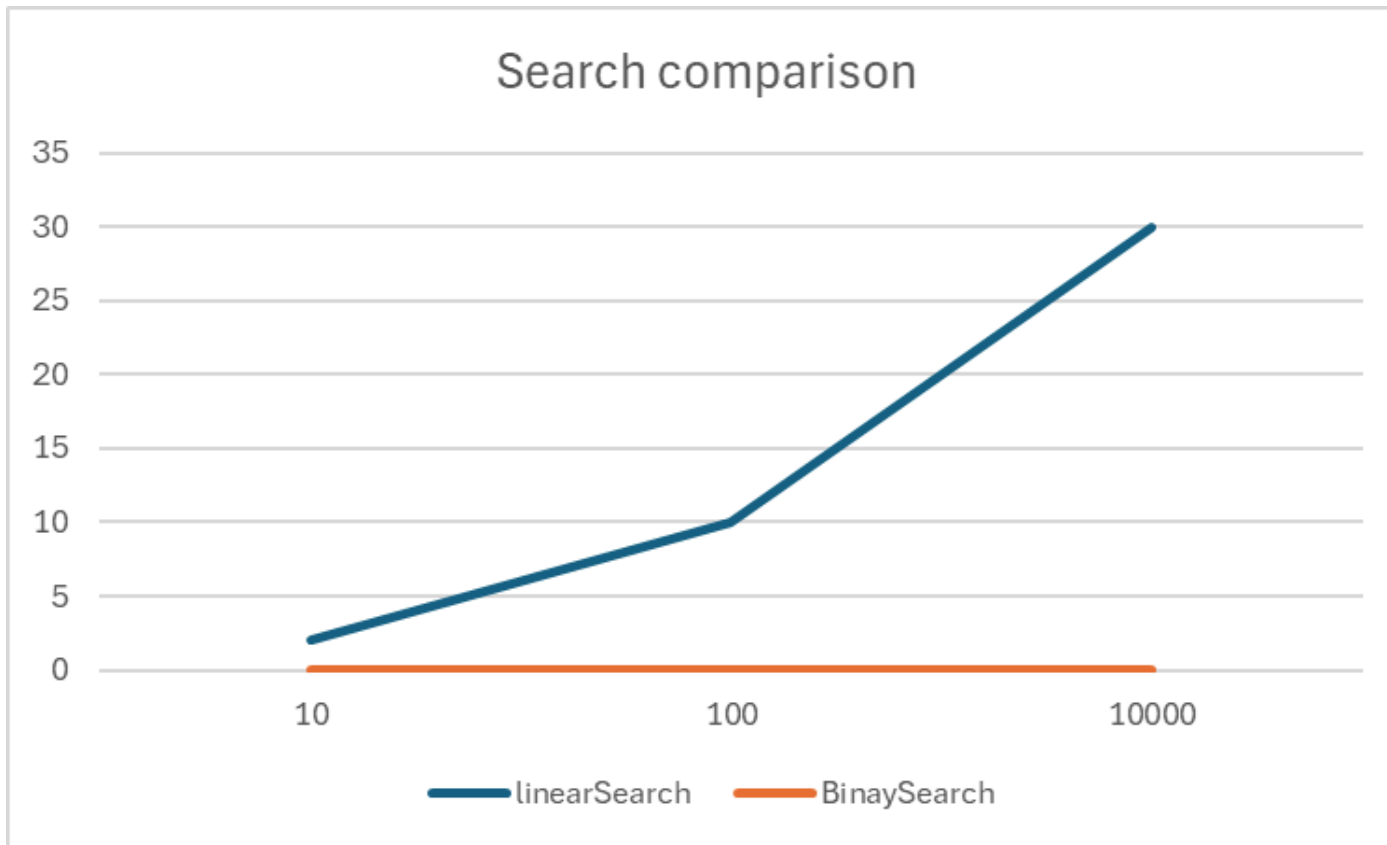
20

Binary Search:

1

2

3



📌 How do the plot?

Linear Search:

1

4

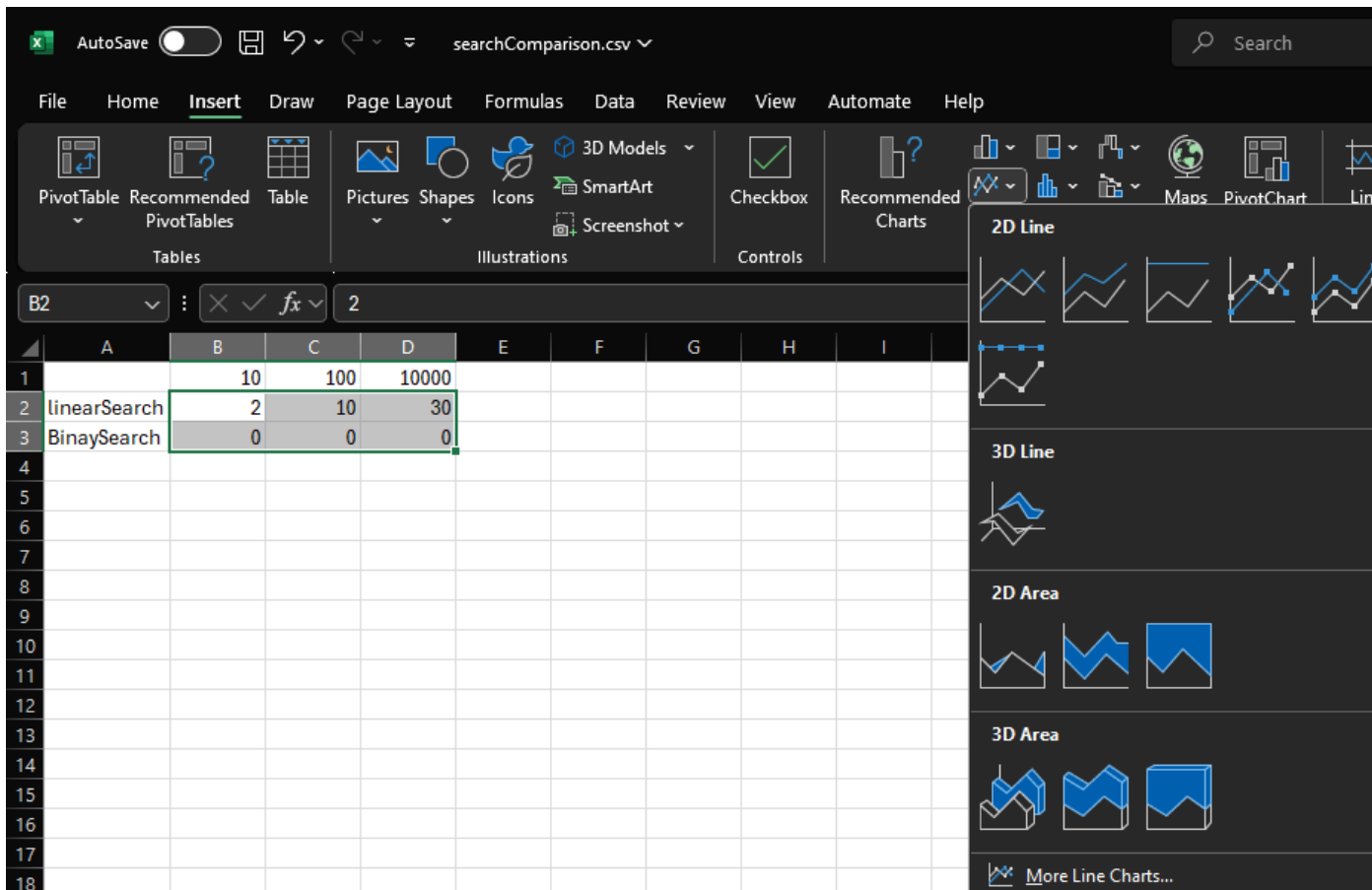
20

Binary Search:

1

2

3



in-class quiz 3

Week9: Given an array of 10,000 elements, how does the number of comparisons made by Binary Search differ from Linear Search in the worst case?

- Binary Search makes at most 14 comparisons, while Linear Search makes 10,000 comparisons.
- Binary Search makes at most 1,000 comparisons, while Linear Search makes 10,000 comparisons.
- Binary Search makes 10,000 comparisons, while Linear Search makes at most 14 comparisons.
- Binary Search and Linear Search both make 10,000 comparisons.

Why measure time is useful?

Time measurement helps developers to:

- Identify bottlenecks in algorithms.
- Compare multiple implementations.
- Ensure scalability for large input sizes.
- Guide optimization strategies.

 For coursework 2, you will perform similar analyses for sorting algorithms (Bubble Sort and Merge Sort)

Final Remarks

Review our Learning Goals

What Algorithms Time Complexity Using Big-O Notation?

- Big-O Notation is a useful tool for measuring an algorithm's complexity

How to measure time of programs in Java?

```
long startTime = System.currentTimeMillis();  
algorithm(input);  
long takenTime = System.currentTimeMillis() - startTime;
```

What is Linear Search?

- A simple search algorithm, which has $O(n)$ complexity

What is Binary Search?

- A search algorithm over a sorted array, which has $O \log(n)$ complexity

DEPARTMENT OF COMPUTER SCIENCE



University of
Reading

Thank you! 😊

Do you have any questions 🙋 ?

The next lecture will cover: Sorting Algorithms